

Week 3 · Exercise 2 — The Token Bill: Measure It, Then Feel It

Transformers, LLMs, RAG and Agents: From Theory to Production

Full name	Marc Masdeu Pons
Partner	Worked solo
Repository	https://github.com/mmasdeups/engsoft-llms (week3/token-bill)

Setup: Windows + WSL2 (Ubuntu), RTX 5080. Every LLM call in both parts is routed through a single LiteLLM proxy that logs per-request token usage to JSONL via a custom callback. Part 1 runs on local **qwen3:14b** (Ollama); Part 2 runs on **gemini-2.5-flash** (Google AI Studio) so both roads share one tokenizer. Code, configs and raw logs are in the repository.

Part 1 — The registry tax

Each number below is the **prompt_tokens of the first LLM call** of a run, captured by the meter. Runs 2–4 all use the same MCP agent (`agent_with_mcp.py`); only the mounted tool registry changes between them, so those three are strictly comparable. The baseline uses a different demo, so it is a reference floor rather than an exact operand.

Run	Configuration	prompt_tokens
1	Baseline, no MCP (2 built-in tools)	274
2	Server mounted, and used	162
3	Server mounted, and NOT used	157
4	Unused, registry grown to 6 tools	944

Registry cost isolated within the same agent: run 3 vs run 4 = $944 - 157 = 787$ tokens for 5 added tools (~157 tokens/tool), on a request that called no tool and returned the same answer either way.

Why does the unused server still cost tokens? An MCP server's registry — every tool's name, description and JSON schema — is injected at the head of the model's context on *every* request, because it is part of what the model must read to know what is available. It is therefore paid in full whether or not any tool is ever called: runs 3 and 4 answered the identical “capital of France” question with zero tool calls, yet cost 157 then 944 tokens purely because the mounted registry grew from one tool to six. Scaling that up — $944 \text{ tokens} \times 50 \text{ turns} \times 10 \text{ such servers} \approx 472,000$ prompt tokens spent before a single useful question is answered.

Part 2 — Same task, two roads

The task: an agent posts a message to a forum, once through a CLI, once through a browser. UPC Àrtemis is behind institutional SSO with the Moodle web-services token flow disabled for students (a correct-credentials call to `login/token.php` returned `invalidlogin`, and no security-keys token is exposed in the profile). Both roads were therefore run against a small local stand-in forum I wrote — a token-gated JSON endpoint for the CLI, an HTML form for the browser — so the CLI-vs-browser token comparison, which is the point of the exercise, is unaffected. Both agents reach **gemini-2.5-flash** through the same proxy and meter.

Road	Tool	LLM calls	Total tokens
CLI	one-line curl to the JSON API	4	4,334
Browser	Playwright MCP driving the HTML form	6	56,580

Ratio $\approx 13\times$ ($56,580 / 4,334$). Both posts landed on the same forum: post #1 by `cli-agent`, and post #2 by `browser-agent` as a reply to #1 (`reply_to: 1`), continuing the conversation. Screenshot of both posts included below.

Why is the browser road so much more expensive? The browser agent perceives the page as an accessibility snapshot — the full structured tree of the DOM — and that snapshot is re-sent into the model's context at every step of the interaction (navigate, read, type, submit). The very first browser call already cost 10,269 prompt tokens — more than the entire CLI road — before any form was touched, and each subsequent step carried another full snapshot (~10.5k–11.8k tokens each). The CLI carries only the token and the message. Notably this is the *cheap* version of the browser road: the stand-in page is tiny, so against real Moodle chrome the ratio would widen.

One task where the browser road is still the right choice. When there is no reachable API at all. This exercise is the concrete case: Àrtemis sits behind SSO with web services locked down for students, so the token-based CLI road is simply impossible against it — driving the logged-in browser UI is the only way an agent could post there. Whenever the interface *is* the contract (legacy systems, SSO-walled portals, sites with no public API), the browser's cost buys the only door that exists.

Artifacts in the repository

week3/token-bill/ contains: `token_meter.py` (LiteLLM logging callback), `meter_report.py`, the two LiteLLM configs, `server-with-6-tools.py` (Part 1 run 4), `forum_server.py` and the `forum-post` skill (Part 2), the `opencode.json` browser-road config, and the four raw `*.jsonl` meter logs backing every number above.

Numbers are read directly from the committed meter logs; the screenshot below shows the two forum posts on the local stand-in server.